

Zadanie 15

Natalia Włodyka

Grudzień 30, 2025

1 Polecenie zadania

Celem zadania jest znalezienie, za pomocą metody Laguerre'a, wszystkich rozwiązań równania:

$$z^4 + iz^3 - z^2 - iz + 1 = 0$$

2 Metoda Laguerre'a

W celu znalezienia pierwiastków wielomianu $P_n(z)$ stopnia n stosujemy metodę Laguerre'a, zadaną iteracją:

$$z_{i+1} = z_i - \frac{n P_n(z_i)}{P_n'(z_i) \pm \sqrt{(n-1) \left((n-1) [P_n'(z_i)]^2 - n P_n(z_i) P_n''(z_i) \right)}}$$

Gdzie znak w mianowniku wybieramy tak, aby jego moduł był większy. Iterację przerywamy wtedy, gdy $|z_i - z_0| < \varepsilon$, gdzie z_0 to szukany pierwiastek. W celu znalezienia wszystkich pierwiastków, po znalezieniu jednego z nich, obniżamy stopień wielomianu. Nowe współczynniki wielomianu o stopniu $n-1$ v_i wyliczamy ze wzoru, gdzie q_i to współczynniki wielomianu o stopniu n :

$$v_i = q_i + z v_{i-1}, \quad v_0 = q_0$$

Postępujemy tak, dopóki nie dojdziemy do wielomianu stopnia 2, którego pierwiastki wyliczamy za pomocą wzoru na równanie kwadratowe.

3 Schemat Hornera

W celu wyliczenia wartości wielomianu w punkcie przekształcamy wielomian zespolony do postaci zagnieżdżonej, co znacząco skróci rachunki:

$$P(z) = a_0 + z(a_1 + z(a_2 + \dots + z(a_{n-1} + z a_n) \dots))$$

Pierwszą i drugą pochodną obliczamy w tym samym przebiegu, za pomocą wzorów:

$$b_i = a_i + z b_{i-1} \rightarrow P(z_0) = b_n$$

$$c_i = b_{i-1} + z c_{i-1} \rightarrow P'(z_o) = c_n$$

$$d_i = c_{i-1} + z d_{i-1} \rightarrow P''(z_o) = 2d_n$$

4 Kod w Pythonie

```

1 import numpy as np
2 from numpy.typing import NDArray
3
4 vector = NDArray[np.complex128]
5 numb = np.complex128
6
7 def horner(coefficients : vector, z : numb) -> tuple[numb, numb,
8     numb]:
9     P = coefficients[0]
10    P_prime = 0.0
11    P_2prime = 0.0
12
13    for i in range(1, len(coefficients)):
14        P_2prime = P_prime + z * P_2prime
15        P_prime = P + z * P_prime
16        P = coefficients[i] + z * P
17
18    return P, P_prime, 2 * P_2prime
19
20 def laguerrea(coefficients : vector, z : numb) -> tuple[int, numb]:
21     epsilon = 1e-9
22     iterations = 1000
23     n = len(coefficients) - 1
24     pom_z = z
25
26     for i in range(iterations):
27         P, P_prime, P_2prime = horner(coefficients, pom_z)
28         numerator = n * P
29         root = (n - 1) * (((n - 1) * P_prime ** 2) - numerator *
30             P_2prime)
31         denominator_minus = P_prime - np.sqrt(root)
32         denominator_plus = P_prime + np.sqrt(root)
33         if abs(denominator_plus) > denominator_minus:
34             denominator = denominator_plus
35         else:
36             denominator = denominator_minus
37
38         next_z = pom_z - numerator/denominator
39
40         if abs(pom_z - next_z) < epsilon:
41             return i + 1, next_z
42
43         pom_z = next_z
44
45     return iterations, pom_z
46
47 def get_new_coefccients(coefficients : vector, z : numb) -> vector
48 :
49     n = len(coefficients) - 1
50     new_coefci = np.zeros(n, dtype=numb)

```

```

48     new_coeffi[0] = coefficients[0]
49     for i in range(1, n):
50         new_coeffi[i] = coefficients[i] + z * new_coeffi[i - 1]
51
52     return new_coeffi
53
54
55 def main():
56     coefficients : vector = np.array([1, 1j , -1, -1j, 1], dtype=
57         numb)
58     roots = []
59     iterations = []
60     while len(coefficients) > 3:
61         z_0: numb = 0 + 0j
62         iteration, root = laguerrea(coefficients, z_0)
63
64         roots.append(root)
65         iterations.append(iteration)
66
67         coefficients = get_new_coeficcients(coefficients, root)
68
69     quad_roots = np.roots(coefficients)
70     roots.extend(quad_roots)
71
72     print("Pierwiastki wielomianu: ")
73     for root in roots:
74         print(root)
75
76     print(iterations)
77
78 if __name__ == "__main__":
79     main()

```

5 Pierwiastki wielomianu

Znalezienie pierwiastki wielomianu za pomocą metody Laguerre'a:

$$z_0 = -0.5877852522924731 - 0.8090169943749475i$$

$$z_1 = -0.9510565162951535 + 0.30901699437494745i$$

$$z_2 = 0.5877852522924728 - 0.8090169943749473i$$

$$z_3 = 0.9510565162951533 + 0.30901699437494756i$$